



Funciones

Python Básico | ⌚ 32 minutos

Qué son?

Las `funciones` o `métodos` (*una definición de función un poco mas especifica que veremos después*) nos permiten crear bloques de código reutilizables.

Nos permiten crear grupos de instrucciones que podemos ejecutar luego con una sola línea cuantas veces queramos.

Ya hemos usado muchas funciones como `print` o `input` pero ahora vamos a crear nuestras propias funciones.

Se crean usando la palabra `def` seguido de su *identificador (nombre)*, un par de paréntesis y dos puntos:

Para su identificador, usamos el mismo estándar que las variables: el

`snake_case`

Por ejemplo, puedo crear una función que llame un `print` para imprimir `Hello World` y `Mi primera funcion`. A esta función la llamaré `print_hello_world`:

```
1 def print_hello_world():  
2     print("Hello World!")  
3     print("Mi primera funcion")
```

Copiar

Si ejecutamos esto, no pasará nada.

Esto es porque solo estamos creando o *definiendo* la función, pero nunca la estamos ejecutando.

Para ejecutarla, o "llamarla", es exactamente igual que hacer un `print`: usando su *identificador* seguido de paréntesis. Es decir `print_hello_world()`:

```
1 def print_hello_world():  
2     print("Hello World!")  
3     print("Mi primera funcion")  
4  
5  
6 print_hello_world()  
7 print_hello_world()
```

Copiar

(va a imprimirlo dos veces porque la estamos llamando dos veces)



Hello World!
Mi primera funcion
Hello World!
Mi primera funcion

Las funciones pueden tener cualquiera de las instrucciones que hemos estado usando hasta ahora.

Incluso pueden tener sus propias variables y llamar a otras funciones.

Es decir que podemos tomar cualquier código que hayamos escrito y convertirlo en una función.

Antes (sin funciones)

```
1 worked_hours = int(input("Ingrese sus horas trabajadas: "))  
2 hour_rate = int(input("Ingrese su tarifa por hora: "))  
3  
4 salary = worked_hours * hour_rate  
5  
6 print(f'Su salario sera de {salary}')
```

Copiar

Después (convertido a función)

```
1 def calculate_salary():  
2     worked_hours = int(input("Ingrese sus horas trabajadas: "))  
3     hour_rate = int(input("Ingrese su tarifa por hora: "))  
4  
5     salary = worked_hours * hour_rate  
6  
7     print(f'Su salario sera de {salary}')
```

Copiar

```
8  
9  
10 calculate_salary()
```

Un código puede tener cantidad de funciones distintas.

Y estas se pueden llamar entre sí.

Lo mejor es siempre definir las funciones arriba del archivo, separadas por 2 líneas en blanco.

Y ordenarlas en el orden en que se ejecutan.

Mal ordenado

```
1 def function_2():  
2     print('Ejecutando 2')  
3 main()  
4  
5 def main():  
6     function_1()  
7     function_2()  
8     function_3()  
9  
10 def function_1():  
11     print('Ejecutando 1')  
12 def function_3():  
13     print('Ejecutando 3')
```

Copiar

Bien ordenado

```
1 def function_1():
2     print('Ejecutando 1')
3
4
5 def function_2():
6     print('Ejecutando 2')
7
8
9 def function_3():
10    print('Ejecutando 3')
11
12
13 def main():
14     function_1()
15     function_2()
16     function_3()
17
18
19 main()
```

Copiar

Parámetros y Retornos

Todas las funciones tienen sus propios **inputs** y **outputs** (entradas y salidas).

Es decir que pueden recibir datos y devolver otros datos.

Parecido a una licuadora a la que le **echas** frutas y leche, y te **devuelve** un batido.

A sus **inputs** les llamamos **parámetros**.

A sus **outputs** les llamamos **retornos**.

Esta es la sintaxis de ambos (**y ya la veremos más a fondo**):

```
1 def nombre(parameter1, parameter2, etc):
2     # instruccion 1
3     # instruccion 2
4     # instruccion 3
5     return output
```

Copiar

Parámetros

Al definir `parámetros` estamos definiendo *variables "placeholder"* a las que les daremos un valor distinto cada vez que ejecutemos la función.

Al ejecutarla, estos valores se establecen dentro de los paréntesis en el orden en que los definimos.

Por ejemplo, creemos una función que imprima 3 `parámetros`:

```
1 def print_parameters(parameter_1, parameter_2, parameter_3):  
2     print(f'This is parameter 1: {parameter_1}')  
3     print(f'This is parameter 2: {parameter_2}')  
4     print(f'This is parameter 3: {parameter_3}')  
5  
6  
7 print_parameters(50, 'Hello', True)
```



This is parameter 1: 50
This is parameter 2: Hello
This is parameter 3: True

Ahora intentemos ejecutarla varias veces dándole distintos valores a sus

`parámetros`:

```
1 def print_parameters(parameter_1, parameter_2, parameter_3):  
2     print(f'This is parameter 1: {parameter_1}')  
3     print(f'This is parameter 2: {parameter_2}')  
4     print(f'This is parameter 3: {parameter_3}')  
5  
6  
7 print_parameters(50, 'Hello', 90)  
8 print('-')  
9 print_parameters([4, 5, 6], 'World', 'Inside')  
10 print('-')  
11 print_parameters('A', 'Function', True)
```



This is parameter 1: 50
This is parameter 2: Hello
This is parameter 3: 90

This is parameter 1: [4, 5, 6]
This is parameter 2: World
This is parameter 3: Inside
-
This is parameter 1: A
This is parameter 2: Function
This is parameter 3: True

Parámetros opcionales

Otro detalle útil es que podemos definir **parámetros opcionales** en las funciones.

Es decir, hacer que ciertos **parámetros** sean opcionales y tengan un valor por defecto en caso de que no se especifiquen a la hora de ejecutarlas.

Estos **parámetros** opcionales deben ir **después** de los requeridos.

Esto lo hacemos justo como al darle un valor a una variable.

```
1 def print_sum_of_numbers(number_a, number_b=5):  
2     print(number_a + number_b)  
3  
4  
5 print_sum_of_numbers(4)
```

[Copiar](#)

El `number_a` será 4, y el `number_b` será 5 porque no le dimos un valor.



9

Retornos

La palabra **return** nos permite decidir qué valor o valores serán el output de la función.

Pueden haber varios **return** en una función, pero **el primero que se ejecute es el que terminará la función**. Nada de lo que haya después de él se ejecutará.

Parecido a un **break** en un ciclo.

Estos outputs los podemos **guardarlos en variables** o usarlos como valores comunes y corrientes.

Por ejemplo, cuando guardamos el **output** de un **input** en una variable.

Por ejemplo, podemos crear una función que sume tres números y retorne su resultado:

```
1 def sum_three_numbers(number1, number2, number3):  
2     return number1 + number2 + number3  
3  
4  
5 result = sum_three_numbers(600, 700, 800)  
6 print(result)
```

[Copiar](#)

O podemos crear una función que encuentre el máximo de 2 números y lo retorne:

```
1 def get_max_of_two_numbers(number1, number2):  
2     if number1 > number2:  
3         return number1  
4  
5     return number2  
6  
7  
8 print(get_max_of_two_numbers(3, 7))
```

[Copiar](#)

✓ 7
13
6

Esta función tiene **2 parámetros**, **number1** y **number2**.

Si **number1** es mayor a **number2**, retorna **number1**.

El **return** hace que la función termine. Su output sale y el resto no se ejecuta. Por eso es que no es necesario el **else** en este caso. Sabemos que si la primera condición se cumple, se ejecuta el **return** y el segundo **return** nunca se ejecutará.

Si no es así, retorna **number2**.

Podemos notar es que estamos encadenando dos funciones, **print** y **get_max_of_two_numbers**.

Esto es porque ya sabemos que esa función va a retornar un valor.

Y una función ejecutada es igual a utilizar un valor o una variable

Podemos usar llamados a funciones como parámetros para otras funciones, ya que el código se ejecutará de adentro hacia afuera.

Es exactamente lo mismo a hacer esto:

```
1 def get_max_of_two_numbers(number1, number2):
2     if number1 > number2:
3         return number1
4
5     return number2
6
7
8 result = get_max_of_two_numbers(4, 15)
9 print(result)
```

[Copiar](#)

15

Otras consideraciones importantes

Algunas funciones no tienen ni **parámetros** ni **retornos**

Como las que vimos al inicio.

Algunas funciones solo **retornan** cosas, pero no tienen **parámetros**.

Algunas funciones solo tienen **parámetros**, pero no **retornan** nada.

Como el **print**. Se encarga de mostrar sus parámetros en pantalla, pero no los retorna.

```
1 print(print('Primer print'))
```

[Copiar](#)

Primer print
None

Al igual que con ejemplo anterior, esto ultimo es igual a hacer:

```
1 result = print('Primer print')
2 print(result)
```

[Copiar](#)



Primer print

None

Nombramiento

Algo muy importante, al igual que con las variables, es darle a las funciones nombres **descriptivos y significativos**.

Esto con el objetivo de que cualquier persona que lea el código pueda entenderlo con facilidad.

Esto ultimo es la base de un código limpio, y es **sumamente importante en ambientes laborales** donde la colaboración es la clave.

Si al nombrar una función tenemos que usar "y", lo mejor es dividir esa función en dos funciones.

Lo mejor es usar **nombres verbales**. Es decir, incluir verbos en el nombre para saber qué acción especifica realiza la función.

Por ejemplo:

Si tenemos una función que *suma* dos números, llamarla `sum_two_numbers`

Si tenemos una función que *genera* una lista de estudiantes, llamarla `generate_student_list`

Usos Principales

El uso principal de las funciones es seguir el principio DRY del código limpio: Don't Repeat Yourself.

Si hay algún procedimiento o patrón que estamos haciendo más de una vez, **lo mejor es convertirlo en una función**.

Así mismo, cuando tenemos un algoritmo muy extenso, **lo mejor es dividirlo en funciones más pequeñas** para mejorar su legibilidad.

Si al nombrar una función tenemos que usar "y", lo mejor es dividir esa función en dos funciones.

Por ejemplo, si tengo un código en el que calculo el promedio de notas de todos varios estudiantes y reviso si se eximieron:

Copiar

```
1 # Scores
2 juan_scores = {
3     "spanish_score": 75,
4     "science_score": 95,
5     "history_score": 54,
6 }
7 sofia_scores = {
8     "spanish_score": 64,
9     "science_score": 56,
10    "history_score": 98,
11 }
12 paul_scores = {
13     "spanish_score": 72,
14     "science_score": 75,
15     "history_score": 79,
16 }
17
18 # Averages
19 juan_scores["average_score"] = (juan_scores["spanish_score"]
20 sofia_scores["average_score"] = (sofia_scores["spanish_score"]
21 paul_scores["average_score"] = (paul_scores["spanish_score"]
22
23 juan_scores["is_exempted"] = juan_scores["average_score"] > 70
24 sofia_scores["is_exempted"] = sofia_scores["average_score"] > 70
25 paul_scores["is_exempted"] = paul_scores["average_score"] > 70
```

Podemos notar que ese calculo de $(\text{spanish_score} + \text{science_score} + \text{history_score}) / 3$ se repite para cada estudiante.

Tambien estamos repitiendo la validación de si el $\text{average_score} > 70$.

Además puede ser confuso y repetitivo de leer ya que estamos haciendo varias cosas a la vez.

Lo mejor dado estas circunstancias es dividir esto en funciones reutilizables más pequeñas.

```
1 def get_average_score(scores):  
2     return (scores["spanish_score"] + scores["science_score"] +  
3  
4  
5 def is_student_exempted(scores):  
6     return scores["average_score"] > 70  
7  
8  
9 # Scores  
10 juan_scores = {  
11     "spanish_score": 75,  
12     "science_score": 95,  
13     "history_score": 54,  
14 }  
15 sofia_scores = {  
16     "spanish_score": 64,  
17     "science_score": 56,  
18     "history_score": 98,  
19 }  
20 paul_scores = {  
21     "spanish_score": 72,  
22     "science_score": 75,  
23     "history_score": 79,  
24 }  
25  
26 # Averages  
27 juan_scores["average_score"] = get_average_score(juan_scores)  
28 sofia_scores["average_score"] = get_average_score(sofia_score  
29 paul_scores["average_score"] = get_average_score(paul_scores)  
30  
31 juan_scores["is_exempted"] = is_student_exempted(juan_scores)  
32 sofia_scores["is_exempted"] = is_student_exempted(sofia_score  
33 paul_scores["is_exempted"] = is_student_exempted(paul_scores)
```

Incluso podemos hacer este código aun mejor convirtiendo estos diccionarios en una lista y haciendo un ciclo que haga lo mismo para todos:

```
1 def get_average_score(scores):  
2     return (scores["spanish_score"] + scores["science_score"] +  
3  
4  
5 def is_student_exempted(scores):  
6     return scores["average_score"] > 70  
7  
8  
9 # Scores  
10 students = [  
11     {  
12         "name": "Juan",  
13         "spanish_score": 75,  
14         "science_score": 95,  
15         "history_score": 54,  
16     },  
17     {  
18         "name": "Sofia",  
19         "spanish_score": 64,  
20         "science_score": 56,  
21         "history_score": 98,  
22     },  
23     {  
24         "name": "Paul",  
25         "spanish_score": 72,  
26         "science_score": 75,  
27         "history_score": 79,  
28     }  
29 ]  
30  
31 # Averages  
32 for student in students:  
33     student["average_score"] = get_average_score(student)  
34     student["is_exempted"] = is_student_exempted(student)  
35     print(student["name"], " is_exempted is ", student["is_exempted"])
```

Puntos Clave

Las funciones son bloques de código reutilizable que pueden aceptar y retornar datos.

Siempre es bueno definir las arriba y dejar dos líneas en blanco entre cada una.

Siempre debemos de usar nombres descriptivos y verbales en nuestras funciones.

Siempre debemos de dividir nuestro código en funciones más pequeñas.